Python Code Challenges Functions

codecademy

Function Parameters

Sometimes functions require input to provide data for their code. This input is defined using *parameters*. *Parameters* are variables that are defined in the function definition. They are assigned the values which were passed as arguments when the function was called, elsewhere in the code.
For example, the function definition defines parameters for a character, a setting, and a skill, which are used as inputs to write the first sentence of a book.

```
def write_a_book(character, setting,
special_skill):
  print(character + " is in " +
        setting + " practicing her " +
        special_skill)
```

Function Indentation

Python uses indentation to identify blocks of code. Code within the same block should be indented at the same level. A Python function is one type of code block. All code under a function declaration should be indented to identify it as part of the function. There can be additional indentation within a function to handle other statements such as `for` and `if` so long as the lines are not indented less than the first line of the function code.

```
# Indentation is used to identify code
blocks

def testfunction(number):
  # This code is part of testfunction
  print("Inside the testfunction")
  sum = 0
  for x in range(number):
    # More indentation because 'for' has
a code block
    # but still part of he function
    sum += x
  return sum
print("This is not part of testfunction")
```

Returning Value from Function

A `return` keyword is used to return a value from a Python function. The value returned from a function can be assigned to a variable which can then be used in the program.
In the example, the function `check_leap_year` returns a string which indicates if the passed parameter is a leap year or not.

```
def check_leap_year(year):
  if year % 4 == 0:
    return str(year) + " is a leap year."
  else:
    return str(year) + " is not a leap
year."

year_to_check = 2018
returned_value =
check_leap_year(year_to_check)
print(returned_value) # 2018 is not a
leap year.
```

Parameters as Local Variables

Function parameters behave identically to a function's local variables. They are initialized with the values passed into the function when it was called.
Like local variables, parameters cannot be referenced from outside the scope of the function.
In the example, the parameter `value` is defined as part of the definition of `my_function`, and therefore can only be accessed within `my_function`.
Attempting to print the contents of `value` from outside the function causes an error.

```
def my_function(value):
  print(value)

# Pass the value 7 into the function
my_function(7)

# Causes an error as `value` no longer
exists
print(value)
```

Multiple Parameters

Python functions can have multiple *parameters*. Just as you wouldn't go to school without both a backpack and a pencil case, functions may also need more than one input to carry out their operations.
To define a function with multiple parameters, parameter names are placed one after another, separated by commas, within the parentheses of the function definition.

```
def ready_for_school(backpack,
pencil_case):
  if (backpack == 'full' and pencil_case
== 'full'):
    print ("I'm ready for school!")
```

Function Arguments

Parameters in python are variables — placeholders for the actual values the function needs. When the function is *called*, these values are passed in as *arguments*.

For example, the arguments passed into the function `.sales()` are the "The Farmer's Market", "toothpaste", and "$1" which correspond to the parameters `grocery_store`, `item_on_sale`, and `cost`.

```
def sales(grocery_store, item_on_sale, cost):
  print(grocery_store + " is selling " + item_on_sale + " for " + cost)

sales("The Farmer's Market", "toothpaste", "$1")
```

Print